

Spring Boot Data & Persistence

In diesem Modul

- JPA/EntityManager, Repository-Pattern
- JPQL, Fetch Joins und EntityGraph
- Projections/DTOs, Auditing, Transaktionen (Propagation/Isolation)

JPA-Architektur

Spring Data legt eine Abstraktionsschicht über den JPA Provider (meist Hibernate).

1. **JPA (Java Persistence API):** Standard-Interfaces wie `EntityManager`.
2. **Hibernate:** Die wichtigste Implementierung.
3. **Spring Data JPA:** Abstraktionsschicht über Hibernate mit Repositories, die Boilerplate-Code reduziert.

Der Entity Manager

Auch wenn wir meistens Repositories nutzen, arbeitet im Hintergrund immer der `EntityManager`.

```
@PersistenceContext
private EntityManager em;

public User save(User user) {
    em.persist(user); // Objekt in den Persistence Context aufnehmen
    return user;
}
```

Der Persistence Context ist ein **First-Level Cache**. Änderungen an Managed Entities werden beim Transaktionsende automatisch in die DB geschrieben ("Dirty Checking").

Repository Pattern

Statt DAOs manuell zu schreiben, definieren wir Interfaces.

```
// Erbt CRUD-Methoden (save, findById, delete...)
public interface UserRepository extends JpaRepository<User, Long> {

    // Derived Query Methods (werden aus dem Methodennamen generiert)
    List<User> findByLastnameAndActiveTrue(String lastname);

    // JPQL Query
    @Query("SELECT u FROM User u WHERE u.email LIKE %:domain")
    List<User> findByEmailDomain(@Param("domain") String domain);
}
```

JPQL – Die Abfragesprache von JPA

- JPQL (*Java Persistence Query Language*) ist eine objektorientierte Abfragesprache, ähnlich zu SQL, aber operiert auf **Entities** und **ihren Attributen** statt auf Tabellen und Spalten.
- Der JPA Provider (z. B. Hibernate) übersetzt JPQL zur Laufzeit in vendor-spezifisches SQL.
- Vorteil: Queries bleiben portabel und eng an das Domain-Modell gekoppelt.

Grundsyntax einer JPQL-Query

```
@Query("SELECT u FROM User u WHERE u.active = true")  
List<User> findActiveUsers();
```

Parametrisierung

```
@Query("SELECT u FROM User u WHERE u.email = :email")  
User findByEmail(@Param("email") String email);
```

Inner Join

```
@Query("SELECT o FROM Order o JOIN o.customer c WHERE c.status = 'PREMIUM'")  
List<Order> findOrdersOfPremiumCustomers();
```

Fetch Join

```
@Query("SELECT u FROM User u JOIN FETCH u.roles")  
List<User> findAllWithRoles();
```

Eager Loading

Das N+1-Problem

Das N+1-Problem beschreibt, was häufig beim Iterieren über Entities passiert, wenn sie selbst eine 1:N-Relation haben.

Man lädt zum Beispiel 100 User in einer Query. Dann greift man auf `user.getAddresses()` zu, welches standardmäßig lazy geschieht.

- Für jeden der 100 User wird ein neues SELECT gefeuert.
- $1 + 100 = 101$ Queries.
- Das ist ein Performance-Killer.

Lösung 1: @EntityGraph

Deklaratives Eager-Loading im Repository.

```
@EntityGraph(attributePaths = {"addresses"})  
List<User> findAll();
```

Lösung 2: JPQL Fetch Join

Lädt ebenfalls den ganzen Graph.

```
@Query("SELECT u FROM User u JOIN FETCH u.addresses")  
List<User> findAllWithAddresses();
```

Projections

Projections mit DTOs

- Projektionen laden nur Teile der Entities.
- Dafür schreibt man neue Klassen, idealerweise als Record, welche die Projektion aufnehmen.
- Dafür gibt es drei Optionen:
 - Interface Projection
 - Class Projection
 - JPQL Projection

Interface Projection

Spring generiert zur Laufzeit einen Proxy.

```
public interface UserView {  
    String getUsername();  
    // Open Projection (SpEL)  
    @Value("#{target.firstname + ' ' + target.lastname}")  
    String getFullName();  
}
```

Class Projection (Records)

Type-safe und performant (selektiert nur benötigte Spalten im SQL).

```
public record UserDto(String username, String email) {}  
// Im Repo:  
List<UserDto> findByActiveTrue();
```

JPQL-Projektion

```
@Query("""  
    SELECT new com.example.UserSummary(u.username, u.email)  
    FROM User u  
    WHERE u.active = true  
    """)  
List<UserSummary> findActiveUserSummaries();
```

Auditing

Automatisches Tracking von Änderungen.

1. `@EnableJpaAuditing` in der Config.
2. Entity anpassen:

```
@EntityListeners(AuditingEntityListener.class)
public class User {
    @CreateDate
    private LocalDateTime createdAt;

    @LastModifiedBy
    private String lastModifiedBy;
}
```


Transaktionsmanagement

Basics

In Spring markiert `@Transactional` Methoden, die atomar ausgeführt werden sollen.

- Default: Rollback nur bei `RuntimeException` (unchecked).
- Checked Exceptions (z.B. `IOException`) lösen standardmäßig **keinen** Rollback aus!
-> `@Transactional(rollbackFor = Exception.class)`

Propagation

Wie verhalten sich Transaktionen bei verschachtelten Service-Aufrufen?

- **REQUIRED (Default):** Nutze vorhandene TX, sonst neue starten.
- **REQUIRES_NEW:** Starte *immer* eine neue TX (pausiere die alte). Wichtig für Logs, die trotz Rollback geschrieben werden sollen.
- **SUPPORTS:** Laufe in TX wenn da, sonst ohne.
- **MANDATORY:** Wirf Exception, wenn keine TX da ist.

```
@Service
public class OrderService {

    private final OrderRepository orders;
    private final AuditService audit;
    private final PaymentService payments;

    @Transactional // REQUIRED als default: gemeinsamer Commit/Rollback
    public void place(Order order) {
        orders.save(order);
        payments.charge(order); // Exception → alles rollt zurück
        audit.logOrder(order);
    }
}
```

```
@Transactional
public void placeWithAuditSafe(Order order) {
    orders.save(order);
}
```

Isolation Levels

- **READ_COMMITTED**: Standard. Verhindert Dirty Reads.
- **REPEATABLE_READ**: Verhindert Non-Repeatable Reads.
- **SERIALIZABLE**: Sperrt Tabellen/Rows aggressiv. Sicher, aber langsam.

```

@Service
public class InventoryService {

    private final ProductRepository products;

    // Sicher gegen Non-Repeatable Reads (z.B. doppelte Reservierung)
    @Transactional(isolation = Isolation.REPEATABLE_READ)
    public void reserve(String sku, int qty) {
        Product p = products.findBySkuForUpdate(sku) // Query mit PESSIMISTIC_WRITE
            .orElseThrow();
        if (p.getStock() < qty) throw new IllegalStateException("Not enough stock");
        p.decreaseStock(qty);
    }

    // Strenger: konsistente Prüfung über mehrere Zeilen (Phantoms vermeiden)
    // Beispiel: Gesamtsumme aller Reservierungen darf den Bestand nicht übersteigen.
    @Transactional(isolation = Isolation.SERIALIZABLE)
    public void placeBulkOrder(String sku, int requested) {
        int alreadyReserved = products.sumReservations(sku); // SELECT SUM(...) FROM reservations WHERE sku=?
        Product p = products.findBySkuForUpdate(sku).orElseThrow();
        if (alreadyReserved + requested > p.getStock()) {
            throw new IllegalStateException("Overbooking prevented");
        }
        products.insertReservation(sku, requested); // eigene Tabelle/Row
    }
}

public interface ProductRepository extends JpaRepository<Product, Long> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select p from Product p where p.sku = :sku")
    Optional<Product> findBySkuForUpdate(@Param("sku") String sku);

    @Query("select coalesce(sum(r.qty),0) from Reservation r where r.sku = :sku")
    int sumReservations(@Param("sku") String sku);

    @Modifying
    @Query("insert into Reservation(sku, qty) values (:sku, :qty)")
    void insertReservation(@Param("sku") String sku, @Param("qty") int qty);
}

```

Advanced-Themen zu NoSQL

MongoDB: Optimistic Locking

Verhindert "Lost Updates" in verteilten Systemen ohne harte DB-Locks.

```
@Document
public class Product {
    @Id String id;
    @Version Long version; // Spring Data prüft und inkrementiert dies
}
```

Wenn zwei User gleichzeitig speichern, gewinnt der erste. Der zweite bekommt eine `OptimisticLockingFailureException`.

Redis als Cache

Caching beschleunigt Lesezugriffe dramatisch.

```
@Service
public class PricingService {

    @Cacheable(value = "prices", key = "#productId")
    public BigDecimal getPrice(String productId) {
        // Teure Berechnung oder DB-Call
        return calculatePrice(productId);
    }

    @CacheEvict(value = "prices", key = "#productId")
    public void updatePrice(String productId, BigDecimal newPrice) {
        // Update Logik
    }
}
```


Distributed Locks (ShedLock)

Szenario: Eine `@Scheduled` Methode soll in einem Cluster (3 Instanzen) nur **einmal** ausgeführt werden.

Lösung: ShedLock (nutzt DB oder Redis als Lock-Provider).

```
@Scheduled(cron = "0 0 * * * *")
@SchedulerLock(name = "dailyReport", lockAtMostFor = "10m")
public void generateDailyReport() {
    // Läuft garantiert nur auf einer Instanz gleichzeitig
}
```